

Transformada Discreta de Fourier

Discrete Fourier Transform (DFT)

Juan Celis

Universidad de Pamplona
Programa de Física

Computational Physics, Cap. 7

- 1 Motivación
- 2 Derivación de la DFT
- 3 Exactitud y Transformada Inversa
- 4 Propiedades para señales reales
- 5 Implementación en Python

Motivación: ¿Por qué la DFT?

Problema central

Dada una función $f(x)$ **solo conocida en puntos discretos** (muestras), ¿cómo calcular sus coeficientes de Fourier?

- La integral de Fourier puede ser **imposible** analíticamente.
- En experimentos reales $f(x)$ se mide a intervalos regulares (señales de audio, sensores, telescopios...).
- La DFT es una herramienta para **descomponer** cualquier señal en sus frecuencias.
- Permite filtrar ruido, detectar periodicidades y resolver ecuaciones diferenciales.

Aplicaciones en física

- Análisis de señales sísmicas.
- Procesamiento de imágenes (astronomía, microscopio electrónico).
- Detección de periodicidades solares.
- Resolución de la ecuación de difusión y de Schrödinger.
- Filtrado y suavizado de datos ruidosos.

Analoga

El analizador de espectro de un equipo de sonido es una DFT en tiempo real.

Derivación de la DFT

Punto de partida: coeficientes de Fourier

Función periódica de período L :

$$\gamma_k = \frac{1}{L} \int_0^L f(x) e^{-i2\pi kx/L} dx$$

Regla trapezoidal con N muestras $x_n = nL/N$:

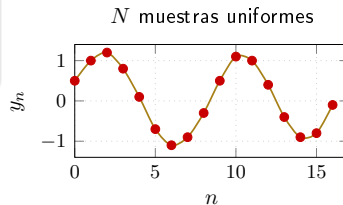
$$\gamma_k \approx \frac{1}{N} \sum_{n=0}^{N-1} y_n e^{-i2\pi kn/N}$$

Convención: la DFT ($c_k = N\gamma_k$)

$$c_k = \sum_{n=0}^{N-1} y_n e^{-i2\pi kn/N} \quad 0 \leq k \leq N-1$$

Muestras y notación

- N muestras igualmente espaciadas:
 $y_n = f(x_n)$
- c_k : coeficientes de Fourier discretos
- Solo se necesitan conocer las y_n y el total N
- Posición x_n y ancho L *no entran* en la fórmula final



Resultado clave

Serie geométrica con $a = e^{i2\pi m/N}$:

$$\sum_{k=0}^{N-1} e^{i2\pi km/N} = \begin{cases} N & m = 0 \text{ o múlt. de } N \\ 0 & \text{otro} \end{cases}$$

Propiedad notable

Aunque los c_k se obtienen con la regla trapezoidal (aproximada), la IDFT recupera las y_n de forma **exacta** (salvo error de redondeo). Las muestras y los coeficientes son representaciones equivalentes del mismo dato.

Transformada inversa (IDFT)

$$y_n = \frac{1}{N} \sum_{k=0}^{N-1} c_k e^{+i2\pi kn/N}$$

Recupera **exactamente** las y_n a partir de los c_k .

Limitación importante

La IDFT solo da los valores *en los puntos de muestra*. No dice nada sobre $f(x)$ entre ellos. Dos funciones muy distintas entre muestras tienen la **misma DFT**.

Simetría de conjugación

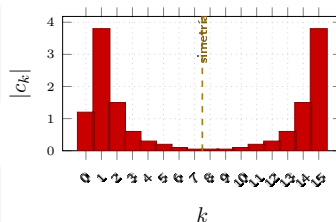
Si las y_n son **reales**: $c_{N-r} = c_r^*$ ($1 \leq r < N/2$)

Es decir: $c_{N-1} = c_1^*$, $c_{N-2} = c_2^*$, etc.

Reducción del cálculo

Solo c_k con $0 \leq k \leq N/2$: total = $\lfloor N/2 \rfloor + 1$
($N//2+1$ en Python). Los demás son conjugados.

Espectro DFT (señal real, $N = 16$)



Señales complejas

Si las y_n son complejas, la simetría NO se cumple y hay que calcular los N coeficientes completos.

Código: dft.py (Newman)

```
from numpy import zeros
from cmath import exp, pi
def dft(y):
    N = len(y)
    c = zeros(N//2+1, complex)
    for k in range(N//2+1):
        for n in range(N):
            c[k] += y[n]*exp(-2j*pi*k*n/N)
    return c
# -- Uso --
from numpy import loadtxt
from pylab import plot, xlim, show
y = loadtxt("pitch.txt", float)
c = dft(y)
plot(abs(c)); xlim(0,500); show()
```

Notas del código

- Solo $N/2 + 1$ coefs. (señales reales).
- `cmath.exp` soporta complejos.
- Complejidad: $\mathcal{O}(N^2)$.
- N grande $\rightarrow$ **FFT**: $\mathcal{O}(N \log N)$.
- Graficar $|c_k|$: espectro de amplitudes.

Limitación

Traducción directa, útil didácticamente pero lenta. En la práctica: `numpy.fft.fft`.

¡Gracias!

Transformada Discreta de Fourier

Basado en: Mark Newman, *Computational Physics*

Universidad de Pamplona — Programa de Física